Formal Verification and Interactive Theorem Proving in Lean 4

Personal Research & Reference Manual

Chan Kwan Ho

October 24, 2026

Abstract

This report details the formalization of calculus and algebraic theorems in the Lean 4 interactive theorem prover, focusing on numerical integration and error estimation. Leveraging Lean's `mathlib` library—specifically its continuous integral and polynomial theories—this work formally verifies the exactness of Simpson's Rule for polynomials of degree up to three. Furthermore, it documents the technical challenges and optimization strategies involved in formal proof construction. By transitioning from verbose manual deductions to idiomatic Lean 4 formulations—such as utilizing `fun_prop` for derivative automation and refining applications of the Fundamental Theorem of Calculus—the codebase achieves significant reductions in proof length and complexity. This document serves as both a record of the specific formalization process and a reference manual for modern interactive theorem proving techniques.

Contents

Abstract i

Abstract iii

1 Introduction 1

2 Theoretical Foundations 2

2.1 Main Proof of Simpson Error Estimation 2

3 Lean 4 Formalization 4

3.1 Base Imports and Configuration 4

3.2 Basic Rules for Simpson Sum 4

4 Formalization Insights & Technical Challenges 6

4.1 Evaluating Polynomials: `eval_eq_sum_range` vs `eval_eq_sum_range'` 6

4.2 Theorem: `simpson_exact_of_degree_le_three` ($\sim$40 $\rightarrow$ 17 lines) . . 7

4.3 Optimizing the FTC Application (`deriv_int_symm`: 85 $\rightarrow$ 21 lines) . 9

4.4 Optimizing Derivative (`simpsonAuxG_deriv_1`: 64 $\rightarrow$ 8 lines) 11

4.5 Optimizing lemma `deriv2_symm` : 15 $\rightarrow$ 5 lines 15

4.6 Optimizing lemma `simpsonAuxG_deriv_2` : 46 $\rightarrow$ 9 lines 16

5 New things learnt 17

5.1 Lean 4 Calculus Convension: `HasDerivAt` Naming Conventions 17

5.1.1 Transitioning: Differentiability vs. Explicit Derivatives 17

5.1.2 Base Rules (Atomic Building Blocks) 18

5.1.3 Calculus Combinators and Operators 18

5.1.4 Scaling and Linear Shifting Extensions 19

5.1.5 Eta-Expanded Suffixes (`s-` Prefix Forms) 19

5.1.6 Pipelining Example 19

6 Conclusion & Future Work 20

6.1 Future Work 20

Abstract

This report details the formalization of calculus and algebraic theorems in the Lean 4 interactive theorem prover, focusing on numerical integration and error estimation. Leveraging Lean's `mathlib` library—specifically its continuous integral and polynomial theories—this work formally verifies the exactness of Simpson's Rule for polynomials of degree up to three. Furthermore, it documents the technical challenges and optimization strategies involved in formal proof construction. By transitioning from verbose manual deductions to idiomatic Lean 4 formulations—such as utilizing `fun_prop` for derivative automation and refining applications of the Fundamental Theorem of Calculus—the codebase achieves significant reductions in proof length and complexity. This document serves as both a record of the specific formalization process and a reference manual for modern interactive theorem proving techniques.

Chapter 1

Introduction

This report documents the journey of formalizing Simpson's Rule and its associated error estimation in the Lean 4 theorem prover. The primary objective of this project is to construct a rigorous, machine-verified proof of the Simpson error bound and ultimately contribute these theorems to `mathlib`, Lean's mathematical library.

Formalizing numerical analysis theorems requires bridging the gap between classical calculus (such as the Fundamental Theorem of Calculus, Rolle's Theorem, and the Mean Value Theorem) and the highly structured, typed environment of Lean 4. Throughout this process, a significant portion of the work was dedicated to not just writing the proofs, but optimizing them. A major theme of this report is the evolution of the codebase: refining proofs to be shorter, more elegant, and more idiomatic to the mathlib standards. By leveraging powerful Lean tactics such as `fun_prop`, `simpa`, and efficient `Finset` manipulations, several theorems were drastically reduced in length and complexity, moving this work closer to the high standards required for a mathlib pull request (PR).

Chapter 2

Theoretical Foundations

2.1 Main Proof of Simpson Error Estimation

Suppose that $f \in C^4(\mathbb{R})$ is a real-valued function. We aim to prove that there exists $\zeta \in [a, b]$ such that:

$$\int_a^b f(x)\,dx - \frac{b-a}{6}\left(f(a) + 4f\left(\frac{a+b}{2}\right) + f(b)\right) = -\frac{(b-a)^5}{2880} f^{(4)}(\zeta)$$

Let $m = \frac{a+b}{2}$ and $h = \frac{b-a}{2}$, such that $[m-h, m+h] = [a, b]$. We define an auxiliary function $G : [0, h] \to \mathbb{R}$ as follows:

$$G(t) = \int_{m-t}^{m+t} f(x)\,dx - \frac{t}{3}\big(f(m-t) + 4f(m) + f(m+t)\big)$$

Since $t \in [0, h]$, the interval $[m-t, m+t]$ is a subset of $[m-h, m+h] = [a, b]$. Thus, G is well-defined on $[0, h]$. Because $f \in C^4([a, b])$, f is continuous on $[m-t, m+t]$; hence, the integral exists for all $t \in [0, h]$. By the Fundamental Theorem of Calculus (FTC), $G \in C^4([0, h])$, and $G(h)$ represents the total Simpson error.

Next, we define another auxiliary function $F(t) = G(t) - \frac{G(h)}{h^5} t^5$. Clearly, $F : [0, h] \to \mathbb{R}$ and $F \in C^4([0, h])$.

Taking the derivatives of $G(t)$, we obtain:

$$\begin{aligned}
G'(t) &= \frac{2}{3}\big(f(m+t) + f(m-t) - 2f(m)\big) - \frac{t}{3}\big(f'(m+t) - f'(m-t)\big) \\
G''(t) &= \frac{1}{3}\big(f'(m+t) - f'(m-t)\big) - \frac{t}{3}\big(f''(m+t) + f''(m-t)\big) \\
G'''(t) &= -\frac{t}{3}\big(f'''(m+t) - f'''(m-t)\big)
\end{aligned}$$

It is straightforward to verify that $G(0) = G'(0) = G''(0) = 0$.

Consequently, $F(0) = G(0) - 0 = 0$ and $F(h) = G(h) - G(h) = 0$. Since $F \in C^4[0, h]$, its derivative is continuous on $[0, h]$. By Rolle's Theorem, there exists $\zeta_1 \in (0, h)$ such that $F'(\zeta_1) = 0$.

Furthermore, $F'(0) = G'(0) - 0 = 0$. Applying Rolle's Theorem again on $[0, \zeta_1]$,

there exists $\zeta_2 \in (0, \zeta_1)$ such that $F''(\zeta_2) = 0$. Since $F''(0) = 0$, a third application of Rolle's Theorem yields $\zeta_3 \in (0, \zeta_2)$ such that $F'''(\zeta_3) = 0$.

Evaluating $F'''(\zeta_3)$, we have:

$$F'''(\zeta_3) = G'''(\zeta_3) - 60\zeta_3^2 \left(\frac{G(h)}{h^5} \right)$$

Since $G'''(\zeta_3) = -\frac{\zeta_3}{3}\big(f'''(m + \zeta_3) - f'''(m - \zeta_3)\big)$, and $f''' \in C[a, b]$ is continuous on $[m - \zeta_3, m + \zeta_3]$, we can apply the Mean Value Theorem (MVT). Since $0 < \zeta_3 < \zeta_2 < \zeta_1 < h$, there exists $\zeta \in [m - \zeta_3, m + \zeta_3] \subseteq [a, b]$ such that:

$$f'''(m + \zeta_3) - f'''(m - \zeta_3) = 2\zeta_3 f^{(4)}(\zeta)$$

Substituting this back yields:

$$F'''(\zeta_3) = -\frac{2}{3}\zeta_3^2 f^{(4)}(\zeta) - 60\zeta_3^2 \left(\frac{G(h)}{h^5} \right)$$

Since $F'''(\zeta_3) = 0$, we find:

$$-\frac{2}{3}\zeta_3^2 f^{(4)}(\zeta) = 60\zeta_3^2 \left(\frac{G(h)}{h^5} \right) \implies G(h) = -\frac{1}{90} f^{(4)}(\zeta) h^5$$

Recalling that $h = \frac{b-a}{2}$, we arrive at the final error term:

$$\text{Error} = -\frac{(b-a)^5}{2880} f^{(4)}(\zeta)$$

for some $\zeta \in [a, b]$.

Chapter 3

Lean 4 Formalization

3.1 Base Imports and Configuration

To formalize the calculus and algebra required for the error estimation, we rely heavily on the integral theory and polynomial libraries within `mathlib`. The fundamental setup is as follows:

```
import Mathlib.Analysis.SpecialFunctions.Integrals.Basic
open Polynomial
noncomputable section

private lemma integrable_term (c : ℝ) (n : ℕ) (a b : ℝ) :
    IntervalIntegrable (fun x => c * x ^ n) MeasureTheory.
       volume a b := by
  exact (continuous_const.mul (continuous_pow n)).
     intervalIntegrable a b

-- Note: The following lemma became useless after optimizations
private lemma integrable_term_one (c : ℝ) (a b : ℝ) :
    IntervalIntegrable (fun x => c * x) MeasureTheory.volume a
       b :=
  (continuous_const.mul continuous_id).intervalIntegrable a b
```

As the formalization evolved to treat all terms uniformly as $c_i x^i$, `integrable_term_one` was rendered obsolete, showcasing an early example of code reduction.

3.2 Basic Rules for Simpson Sum

The general definition for the Simpson Sum and its associated auxiliary function $G(t)$ are defined as follows:

```
def simpsonSum (f : ℝ → ℝ) (a b : ℝ) : ℝ :=
  (b - a) / 6 * (f a + 4 * f ((a + b) / 2) + f b)
```

```
private def simpsonAuxG (f : ℝ → ℝ) (m : ℝ) (t : ℝ) : ℝ :=
  (∫ x in (m - t)..(m + t), f x) - (t / 3) * (f (m - t) + 4 * f
      m + f (m + t))
```

From here, we developed the common associativity and commutativity properties of the Simpson sum operation over real-valued functions, establishing basic rules such as `lemma simpsonSum_add (f g : ℝ → ℝ) (a b : ℝ)` and `lemma simpsonSum_smul (c : ℝ) (f : ℝ → ℝ) (a b : ℝ)`.

Chapter 4

Formalization Insights & Technical Challenges

A primary goal of this formalization was to refine the proofs to be shorter, more elegant, and more idiomatic to the `mathlib` standards. This chapter documents the evolution of the codebase and the technical challenges overcome.

4.1 Evaluating Polynomials: `eval_eq_sum_range` vs `eval_eq_sum_range'`

To prove the exactness of Simpson's Rule for polynomials of degree ≤ 3, we must partition the polynomial into separate degrees to eliminate the degree sum for each term. We compared two approaches: `rw[p.eval_eq_sum_range]` and `rw[p.eval_eq_sum_range']`.

The former (`range`) can be applied to general polynomials for any degree (finite or infinite). However, it requires a cumbersome proof using `Finset.sum_subset` to handle the coefficients beyond the natural degree. The latter (`range'`) requires supplying a bound $n : \mathbb{N}$ alongside a proof that p.natDegree $< n$, which drastically simplifies the proof:

```
-- Optimized approach using eval_eq_sum_range'
private lemma eval_le_three0 (p : ℝ[X]) (hp : p.natDegree ≤ 3)
   (x : ℝ) :
    p.eval x = p.coeff 0 * 1 + p.coeff 1 * x + p.coeff 2 * x^2
       + p.coeff 3 * x^3 := by
  have h4 : p.natDegree < 4 := by omega
  rw [p.eval_eq_sum_range' h4]
  rw [Finset.sum_range_succ, Finset.sum_range_succ,
      Finset.sum_range_succ, Finset.sum_range_succ]
  ring

-- Original, longer approach using eval_eq_sum_range
private lemma eval_le_three1 (p : ℝ[X]) (hp : p.natDegree ≤ 3)
```

```
  (x : ℝ) :
   p.eval x = p.coeff 0 * 1 + p.coeff 1 * x + p.coeff 2 * x^2
      + p.coeff 3 * x^3 := by
  rw [p.eval_eq_sum_range]
  have h4: p.natDegree + 1 ≤ 4 := by omega
  have h_sum :      i ∈ Finset.range (p.natDegree + 1), p.coeff
     i * x ^ i =
                    i ∈ Finset.range 4, p.coeff i * x ^ i := by
    apply Finset.sum_subset
    · intro i hi
      rw [Finset.mem_range] at hi ⊢
      omega
    · intro i hi hni
      rw [Finset.mem_range] at hi hni
      have hlt : p.natDegree < i := by omega
      rw [Polynomial.coeff_eq_zero_of_natDegree_lt hlt,
         zero_mul]
  rw [h_sum]
  rw [Finset.sum_range_succ, Finset.sum_range_succ,
      Finset.sum_range_succ, Finset.sum_range_succ]
  ring
```

The `eval_le_three1` approach was ultimately discarded in favor of the cleaner `eval_le_three0`.

4.2 Theorem: `simpson_exact_of_degree_le_three` (∼40 → 17 lines)

The original proof of exactness relied heavily on `intervalIntegral.integral_add`, manually chaining integrability proofs for each decomposed polynomial term. By replacing this with `intervalIntegral.integral_finsetSum`, we achieved a massive reduction in code length.

```
-- Original (verbose) implementation (~40 lines)
theorem simpson_exact_of_degree_le_three1 (p : ℝ[X]) (hp : p.
   natDegree ≤ 3) (a b : ℝ) :
    ∫ x in a..b, p.eval x = simpsonSum (fun x => p.eval x) a b
       := by
  simp_rw [eval_le_three0 p hp]
  have hi0 : IntervalIntegrable (fun x => p.coeff 0 * 1)
     MeasureTheory.volume a b :=
    integrable_term (p.coeff 0) 0 a b
```

```
  have hi1 : IntervalIntegrable (fun x => p.coeff 1 * x)
     MeasureTheory.volume a b :=
    integrable_term_one (p.coeff 1) a b
  have hi2 : IntervalIntegrable (fun x => p.coeff 2 * x^2)
     MeasureTheory.volume a b :=
    integrable_term (p.coeff 2) 2 a b
  have hi3 : IntervalIntegrable (fun x => p.coeff 3 * x^3)
     MeasureTheory.volume a b :=
    integrable_term (p.coeff 3) 3 a b
  have hi01 : IntervalIntegrable (fun x => p.coeff 0 * 1 + p.
     coeff 1 * x) MeasureTheory.volume a b :=
    hi0.add hi1
  have hi012 : IntervalIntegrable (fun x => p.coeff 0 * 1 + p.
     coeff 1 * x + p.coeff 2 * x^2) MeasureTheory.volume a b :=
    hi01.add hi2
  rw [intervalIntegral.integral_add hi012 hi3]
  rw [intervalIntegral.integral_add hi01 hi2]
  rw [intervalIntegral.integral_add hi0 hi1]
  sorry

-- Optimized implementation using finsetSum (17 lines)
theorem simpson_exact_of_degree_le_three0 (p : ℝ[X]) (hp : p.
   natDegree ≤ 3) (a b : ℝ) :
    ∫ x in a..b, p.eval x = simpsonSum (fun x => p.eval x) a b
       := by
  have h4 : p.natDegree < 4 := by omega
  simp_rw [p.eval_eq_sum_range' h4]
  rw [intervalIntegral.integral_finsetSum]
  · rw [Finset.sum_range_succ, Finset.sum_range_succ,
       Finset.sum_range_succ, Finset.sum_range_succ]
    ring_nf
    sorry
  · intro i _
    exact integrable_term (p.coeff i) i a b
```

This change split the proof into two goals using the · combinator. The latter goal elegantly resolves integrability by treating all terms uniformly as $c_i x^i$, thereby completely eliminating the need for `eval_le_three0/1` entirely in the main code.

To reduce the Finset, we utilized several key properties:

- `Finset.sum_range_succ`: Peels off the highest term iteratively (e.g., $\sum_{i=0}^{4} f(i) = \sum_{i=0}^{3} f(i) + f(4)$).

- `Finset.range_zero`: Converts the sum over `range 0` into a sum over $\emptyset$, which evaluates to 0.
- `zero_add`: Simplifies $0 + f(0) + f(1) + \ldots$ to $f(0) + f(1) + \ldots$
- `pow_zero` and `pow_one`: Normalizes $x^0 = 1$ and $x^1 = x$.

4.3 Optimizing the FTC Application (`deriv_int_symm`: 85 $\rightarrow$ 21 lines)

When finding the derivative of the integral part of $G(t)$, the original approach manually split the integral, applied `intervalIntegral.integral_hasDerivAt_right`, and dealt with the chain rule for $f(m+t)$ and $f(m-t)$ using 4 separate `have` statements, leading to massive duplication.

The optimized approach was divided into three refined steps:

Part 1: Splitting the Integral

We combined the `ring` and `integral_symm` steps. A major technical hurdle was that `hf.intervalIntegrable _ _` left the measure as an unconstrained metavariable, causing Lean to fail. Explicitly stating `m (m-u)` and `m (m+u)` resolved this:

```
private lemma deriv_int_symm (f : ℝ → ℝ) (m t : ℝ) (hf :
    Continuous f) :
     HasDerivAt (fun t => ∫ x in (m-t)..(m+t), f x) (f (m + t) +
         f (m - t)) t := by
  have h_split : (fun t => ∫ x in (m-t)..(m + t) , f x) =
                 (fun t => (∫ x in m..(m+t), f x) - (∫ x in m
                    ..(m-t), f x)) := by
    ext u
    have h1 : IntervalIntegrable f MeasureTheory.volume (m - u)
        m :=
      hf.intervalIntegrable _ _
    have h2 : IntervalIntegrable f MeasureTheory.volume m (m +
       u) :=
      hf.intervalIntegrable _ _
    rw [← intervalIntegral.integral_add_adjacent_intervals h1
       h2]
    rw [intervalIntegral.integral_symm (m-u) m]
    ring
  rw [h_split]
```

Part 2: The Left and Right Chain Rules

By proving a function continuous everywhere is measurable, we fed the hypotheses

into `integral_hasDerivAt_right` and used `convert` to apply the chain rule via composition:

```
-- The Right part
have h_right_base : HasDerivAt (fun u => ∫ x in m..u, f x) (f
    (m + t)) (m + t) := by
  exact intervalIntegral.integral_hasDerivAt_right
    (hf.intervalIntegrable m (m + t))
    (hf.stronglyMeasurableAtFilter MeasureTheory.volume (nhds
        (m + t)))
    hf.continuousAt
have h_shift_right : HasDerivAt (fun t => m + t) 1 t := (
   hasDerivAt_id t).const_add m
have h_right : HasDerivAt (fun t => ∫ x in m..(m + t), f x) (
   f (m + t)) t := by
  convert h_right_base.comp t h_shift_right using 1
  · funext u; rfl
  · ring

-- The Left Part
have h_left_base : HasDerivAt (fun u => ∫ x in m..u, f x) (f
   (m - t)) (m - t) := by
  exact intervalIntegral.integral_hasDerivAt_right
    (hf.intervalIntegrable m (m - t))
    (hf.stronglyMeasurableAtFilter MeasureTheory.volume (nhds
        (m - t)))
    (hf.continuousAt)
have h_shift_left : HasDerivAt (fun t => m - t) (-1) t := by
  simpa [sub_eq_add_neg, add_comm, add_left_comm, add_assoc]
     using
    ((hasDerivAt_id t).neg.const_add m)
have h_left : HasDerivAt (fun t => ∫ x in m..(m - t), f x) (-
   f (m - t)) t := by
  convert h_left_base.comp t h_shift_left using 1
  · funext u; rfl
  · ring
```

Part 3: Final Resolution using `simpa`

Finally, we used `simpa` to apply the chain rule and simplify the arithmetic directly. `simpa using e` attempts to close the goal by matching the simplified version of `e` with the simplified goal, reducing 20 lines of tedious arithmetic resolution to just 2 lines:

```
have h_total : HasDerivAt
    (fun t => (∫ x in m..(m + t), f x) - (∫ x in m..(m - t),
       f x))
    (f (m + t) + f (m - t)) t := by
  convert h_right.sub h_left using 1
  ring
simpa using h_total
```

4.4 Optimizing Derivative (`simpsonAuxG_deriv_1`: 64 → 8 lines)

Taking the derivative of the auxiliary function $G(t)$ required extensive boilerplate to prove that each arithmetic combination of functions was differentiable. By leveraging the `HasDerivAt, hasDerivAt` tactic, Lean automatically inferred these differentiability constraints (`DifferentiableAt`) → (`Derivative`) constraints, allowing the code to focus purely on the structural application of the product and chain rules. The fully realized optimization for the first derivative of $G(t)$ is shown below (optimized from over 64 lines) into parts:

```
private lemma simpsonAuxG_deriv1 (f : ℝ → ℝ) (m t : ℝ) (hf
   : ContDiff ℝ 1 f) :
deriv (simpsonAuxG f m) t =
  (2 / 3) * (f (m + t) + f (m - t) - 2 * f m) -
  (t / 3) * (deriv f (m + t) - deriv f (m - t)) := by
```

- Part 1: Defining Differentiability and also Continuity

```
      have hf_cont : Continuous f := hf.continuous
have hf_diff : Differentiable ℝ f := hf.differentiable (
   by simp)
```

- Part 2: Constructing partitioned block of Differentiability and Derivative:

```
        have h_int : deriv (fun s => ∫ x in (m - s)..(m +
           s), f x) t =
      f (m + t) + f (m - t) := (deriv_int_symm f m t
         hf_cont).deriv
have h_prod :
    deriv (fun s => (s / 3) * (f (m - s) + 4 * f m + f (
       m + s))) t =
      (1 / 3) * (f (m - t) + 4 * f m + f (m + t))
```

```
      + (t / 3) * (deriv f (m + t) - deriv f (m - t))
        := by
have hl : DifferentiableAt ℝ (fun s => s / 3) t :=
  differentiableAt_id.div_const 3
have hr : DifferentiableAt ℝ (fun s => (f (m - s) + 4
  * f m + f (m + s))) t := by fun_prop
change deriv ((fun s : ℝ => s / 3) *
  (fun s => f (m - s) + 4 * f m + f (m + s))) t = _
rw [deriv_mul hl hr]
rw [deriv_div_const, deriv_id'']
have h_1 :
    deriv (fun s : ℝ => f (m - s) + 4 * f m + f (m + s
      )) t =
    deriv f (m + t) - deriv f (m - t) := by
  have h1 : deriv (fun s : ℝ => f (m - s) ) t = -
    deriv f (m - t) := by
    have h_inner : HasDerivAt (fun s : ℝ => m - s)
      (-1) t := by
      simpa [sub_eq_add_neg, add_comm] using
        (hasDerivAt_id t).neg.const_add m
    have h_comp :=
      (hf_diff (m - t)).hasDerivAt.comp t h_inner
    simpa [Function.comp_def] using h_comp.deriv
  have h2 : deriv (fun s : ℝ => f (m + s)) t = deriv f
    (m + t) := by
    have h_inner : HasDerivAt (fun s : ℝ => m + s) 1 t
      :=
    (hasDerivAt_id t).const_add m
    have h_comp :=
      (hf_diff (m + t)).hasDerivAt.comp t h_inner
    simpa [Function.comp_def] using h_comp.deriv
  have h3 : DifferentiableAt ℝ (fun s : ℝ  => f (m - s
    ) + 4 * f m) t := by fun_prop
  have h4 : DifferentiableAt ℝ (fun s : ℝ  => f (m + s
    )) t := by fun_prop
```

- Part 3: Combining partitioned block via basic differentiation rules:

```
  rw [deriv_fun_add h3 h4]
  rw [deriv_fun_add (by fun_prop) (by fun_prop)]
  simp [h1, h2]
```

```
    ring
  dsimp
  rw [h_1]
unfold simpsonAuxG
rw [← Pi.sub_def]
have h_int_hasDerivAt :
    HasDerivAt (fun s => ∫ x in (m - s)..(m + s), f x)
      (f (m + t) + f (m - t)) t :=
  deriv_int_symm f m t hf_cont
have h_int := h_int_hasDerivAt.deriv
have h1 : DifferentiableAt ℝ
    (fun s => ∫ x in (m - s)..(m + s), f x) t :=
  h_int_hasDerivAt.differentiableAt
have h2 : DifferentiableAt ℝ (fun s ↦ s / 3 * (f (m - s)
    + 4 * f m + f (m + s))) t := by fun_prop
rw [deriv_sub h1 h2]
rw [h_int, h_prod]
ring
```

Where we can see that Part 2 used the most lines of proof and at Part 3, most of the lines go to the redundant `deriv.add` work and also defining and dealing with the derivative of each block like `deriv f(m + t)`,`deriv f(m - t)`, `deriv of f(m-t) + 4 * f(m)` and also chain rules, where they can be simplified by some useful shortcut in Lean4.

From chapter 5, I have learnt that from the hypothesis of Differentiability, let's say `hd : DifferentiableAt ℝ f x`, one can actually extract the derivative out from it via

`hd.hasDerivAt` , to obtain the type `HasDerivAt f (deriv f x) x`

which this is very useful as we do not need to digest the entire function, but also getting the derivative and also we are able to use the differentiation rules to build complex sentence.

The improved code are established using following steps:

1. By the hypothesis of the lemma `(hf : ContDiff ℝ 1 f)`, we are able to get `(hd : Differentiable ℝ f) := hf.Differentiable (by simp)`

2. From here we can extract useful information out from it by

   `(hd (m + t)).hasDerivAt , (hd (m - t).hasDerivAt)`

to get the type

```
HasDerivAt f (deriv f (m ± t)) (m ± t)
```

3. And from here we can abuse the rule of differentiation by using HasDerivAt using fast code

4. Deal with $f'(m+t)$: we need chain rule, and identity, so to suit the type for `HasDerivAt.comp hg x hf`, we use fast code `hg.comp x hf`, where the hypothesis of `hg,hf` can be obtained by `(hd (m + t)).hasDerivAt` and `(hasDerivAt_id t).const_add m` respectively.

5. Hence get $f'(m+t)$:

   ```
   have h1 := (hd (m + t)).hasDerivAt.comp t ((hasDerivAt_id t).const_add m)
   ```

   and similarly that $f'(m-t)$:

   ```
   have h2 := (hd (m - t)).hasDerivAt.comp t ((hasDerivAt_id t).neg.const_add m)
   ```

6. So the derivative of $f(m+t)+4f(m)+f(m-t)$ can be obtained by

   ```
   h3 := (h1.add hasDerivAt_const t 4*f m).add h2
   ```

7. and deriv. of $\frac{t}{3}(f(m+t)+4f(m)+f(m-t))$ is simply just

   ```
   h_prod := ((hasDerivAt_id t).div_const 3).mul h3
   ```

   (or directly combining 6,7 with

   ```
   h_prod :=((hasDerivAt_id t).div_const 3).mul

   (h1.add hasDerivAt_const t 4*f m).add h2))
   ```

Finally we combine the result from the lemma `int_deri_symm`(which also has the type of `HasDerivAt`) with the code line

```
have h_G : HasDerivAt (simpsonAuxG f m) _ t := (deriv_int_symm f m t hf.continuous)
```

and rewriting back to `deriv` from `HasDerivAt` by using `h_G.deriv` and `dsimp , ring` to get desired proof.

Where the whole proof is given by:

```
have hd : Differentiable ℝ f := hf.differentiable (by simp)
have h1 := (hd (m + t)).hasDerivAt.comp t ((hasDerivAt_id t).
   const_add m)
have h2 := (hd (m - t)).hasDerivAt.comp t ((hasDerivAt_id t).
   neg.const_add m)
have h_prod := ((hasDerivAt_id t).div_const 3).mul
  ((h2.add (hasDerivAt_const t (4 * f m))).add h1)
have h_G : HasDerivAt (simpsonAuxG f m) _ t := (
   deriv_int_symm f m t hf.continuous).sub h_prod
rw [h_G.deriv]
dsimp
ring
```

4.5 Optimizing lemma `deriv2_symm` : 15 → 5 lines

Using the same idea as above, we can simplify the derivative of f of order 2 into fast code. The lemma

```
private lemma deriv2_symm (f : ℝ → ℝ) (m t : ℝ) (hff :
   ContDiff ℝ 2 f) :
HasDerivAt (fun s => deriv f (m + s) - deriv f (m - s))
(iteratedDeriv 2 f (m + t) + iteratedDeriv 2 f (m - t)) t
   := by
```

has most of the line dealing with the derivative of $f'(m+t), f'(m-t)$ as follows:

```
  have hdf : Differentiable ℝ (deriv f) := hff.
     differentiable_deriv_two
have hd : Differentiable ℝ f := hff.differentiable (by simp)
have h1 : HasDerivAt (fun s : ℝ => deriv f (m + s))
    (deriv (deriv f) (m + t)) t := by
  have h1i : HasDerivAt (fun s : ℝ => m + s) 1 t :=
    (hasDerivAt_id t).const_add m
  simpa [Function.comp_def] using (hdf (m + t)).hasDerivAt.
     comp t h1i
have h2 : HasDerivAt (fun s : ℝ => deriv f (m-s))
    (-(deriv (deriv f) (m - t))) t := by
  have h2i : HasDerivAt (fun s : ℝ => m - s) (-1) t := by
    simpa [sub_eq_add_neg, add_comm] using
    (hasDerivAt_id t).neg.const_add m
  simpa [Function.comp_def] using (hdf (m - t)).hasDerivAt.
     comp t h2i
```

```
have h_sub := h1.sub h2
simpa [iteratedDeriv_succ, iteratedDeriv_one] using h_sub
```

where the two code

```
  have h1 : HasDerivAt (fun s : ℝ => deriv f (m + s))
   (deriv (deriv f) (m + t)) t := by
 have h1i : HasDerivAt (fun s : ℝ => m + s) 1 t :=
   (hasDerivAt_id t).const_add m
 simpa [Function.comp_def] using (hdf (m + t)).hasDerivAt.
    comp t h1i
```

can be reduced to only 1 single line by

```
have hdf :  Differentiable ℝ (deriv f) := hff.differentiable_deriv_two
```

Where we obtain the differentiability by `hdf`, hence can use the fast code from it.

```
have h1 := (hdf (m + t)).hasDerivAt.comp t ((hasDerivAt_id t)
   .const_add m)
have h2 := (hdf (m - t)).hasDerivAt.comp t ((hasDerivAt_id t)
   .neg.const_add m)
have h_sub := h1.sub h2
simpa [Pi.sub_def, Function.comp_def, iteratedDeriv_succ,
   iteratedDeriv_one] using h_sub
```

to get

```
HasDerivAt (fun s ↦ deriv f (m + s) - deriv f (m - s)) (
   iteratedDeriv 2 f (m + t) + iteratedDeriv 2 f (m - t)) t
```

where

1. `Pi.sub_def` rw $(f - g)s$ to $fs - gs$
2. `Function.comp_def` rw $(f \circ g)s$ to $f(g(s))$
3. `iteratedDeriv_succ / iteratedDeriv_one` convert `iteratedDeriv 2 f` to `deriv (iteratedDeriv 1 f)` and to `deriv (deriv f)`
4. `simpa` apply the simplification to both `h_sub` and also the goal.

4.6 Optimizing lemma `simpsonAuxG_deriv_2 : 46 →` 9 lines

We use the same tactic as in simpsonAuxG_deriv_1.

Chapter 5

New things learnt

5.1 Lean 4 Calculus Convension: `HasDerivAt` Naming Conventions

Lean 4 implements a structured naming convention for calculus derivatives that separates primary base calculations from composition operators (combinators). By leveraging Generalized Dot Notation, complex derivative terms can be pipelined fluidly from left to right instead of wrapping functions from the outside in.

5.1.1 Transitioning: Differentiability vs. Explicit Derivatives

Mathlib structures derivatives into three distinct concepts: predicate existence (`DifferentiableAt`), explicit relational proofs (`HasDerivAt`), and the functional derivative operator (`deriv`).
You can jump between these forms instantly using dot notation variants.

Transition Direction	**Fast Code Syntax**	**Resulting Type**
`HasDerivAt` $\Longrightarrow$ `DifferentiableAt`	`hf.differentiableAt`	`DifferentiableAt ℝ f x`
`DifferentiableAt` $\Longrightarrow$ `HasDerivAt`	`hd.hasDerivAt`	`HasDerivAt f (deriv f x) x`
`HasDerivAt` $\Longrightarrow$ Equation	`hf.deriv`	`deriv f x = f'`

Table 5.1: Differentiability Conversion Methods

- `hf.differentiableAt:` Drops the exact value f' and yields the proof that f can be differentiated at x. Crucial when applying theorems that only demand existence.
- `hd.hasDerivAt:` Takes an existence hypothesis `hd` and constructs a relation specifying that the derivative is exactly equal to the abstract expression `deriv f x`.

- `hf.deriv`: Directly returns a definitional equality proving that the functional evaluation of `deriv f x` equals your local explicit value f'.

5.1.2 Base Rules (Atomic Building Blocks)

Base rules establish core mathematical primitives from scratch. In Lean 4, these functions begin with a **lowercase letter** and require explicit evaluation coordinates (e.g., the point x or constant c).

Mathematical Function	Lean 4 Base Rule	Evaluated Output Type
$f(x) = x$ (Identity)	`hasDerivAt_id x`	`HasDerivAt (fun x => x) 1 x`
$f(x) = c$ (Constant)	`hasDerivAt_const c x`	`HasDerivAt (fun x => c) 0 x`
$f(x) = x^n$ (Power Rule)	`hasDerivAt_pow n x`	`HasDerivAt (fun x => x^n) (n * x^(n-1)) x`

Table 5.2: Lean 4 Primary Base Derivative Rules

5.1.3 Calculus Combinators and Operators

When composing functions, identifiers begin with an **uppercase letter** namespace (`HasDerivAt`). Given hypotheses `hf : HasDerivAt f f' x` and `hg : HasDerivAt g g' x`, operations can be executed via traditional explicit formats or streamlined dot notation.

Operation	Standard Verbose	Fast Code (Dot)	Derivative
Addition	`HasDerivAt.add hf hg`	`hf.add hg`	$f' + g'$
Subtraction	`HasDerivAt.sub hf hg`	`hf.sub hg`	$f' - g'$
Negation	`HasDerivAt.neg hf`	`hf.neg`	$-f'$
Product Rule	`HasDerivAt.mul hf hg`	`hf.mul hg`	$f' \cdot g(x) + f(x) \cdot g'$
Inverse	`HasDerivAt.inv hg`	`hg.inv`	$-g'/g(x)^2$
Quotient Rule	`HasDerivAt.div hf hg`	`hf.div hg`	$(f' \cdot g(x) - f(x) \cdot g')/g(x)^2$
Chain Rule	`HasDerivAt.comp hg x hf`	`hg.comp x hf`	$g' \cdot f'$

Table 5.3: Algebraic Structural Combinators

5.1.4 Scaling and Linear Shifting Extensions

To perform arithmetic modifications using inline constants without declaring standalone constant hypotheses, Lean 4 exposes direct suffix extensions to existing proofs:

- **Constant Addition ($c + f(x)$):** `hf.const_add c` $\Longrightarrow$ `fun x => c + f x`
- **Scalar Shifting ($f(x) + c$):** `hf.add_const c` $\Longrightarrow$ `fun x => f x + c`
- **Constant Multiplier ($c \cdot f(x)$):** `hf.const_mul c` $\Longrightarrow$ `fun x => c * f x`
- **Scalar Scaling ($f(x) \cdot c$):** `hf.mul_const c` $\Longrightarrow$ `fun x => f x * c`

5.1.5 Eta-Expanded Suffixes (s- Prefix Forms)

If target environment goals contain explicit functional abstractions (λ-expressions) instead of canonical composition objects (such as $g \circ f$), appending specific prefixes or suffixes guarantees precise structural unification:

- `hg.scomp x hf` matches the goal `fun x => g (f x)` (instead of $g \circ f$).
- `hf.fun_add hg` matches the goal `fun x => f x + g x` (instead of $f + g$).
- `hf.fun_mul hg` matches the goal `fun x => f x * g x` (instead of $f \cdot g$).

5.1.6 Pipelining Example

A complex function such as $f(x) = 5x^3 + 7$ evaluates via standard code using deeply nested parentheses. Under Fast Code guidelines, it resolves left-to-right following algebraic construction:

```
-- Elegant, linear pipeline matching 5x^3 + 7
example (x : \mathbb{R}) : HasDerivAt (fun x => 5 * x^3 + 7)
(5 * (3 * x^(3 - 1)) + 0) x :=
  ((hasDerivAt_pow 3 x).const_mul 5).add_const 7
```

Chapter 6

Conclusion & Future Work

The process of formalizing the Simpson error estimation in Lean 4 has been an illuminating exercise in both mathematical translation and codebase optimization. What began as a brute-force translation of calculus theorems evolved into a heavily optimized, idiomatic Lean script. Through the strategic use of `Finset` operations, `intervalIntegral.integral_finsetSum`, `simpa`, and `fun_prop`, the proof was condensed significantly—in some lemmas, code length was reduced by over 75%.

6.1 Future Work

Moving forward, the immediate next steps are to:

1. **Complete the Sorry Blocks:** Resolve the remaining `sorry` gaps in the algebraic ring normalizations and bounding theorems.
2. **Generalization:** Ensure the auxiliary definitions (like G and F) are properly enclosed in localized namespaces to avoid polluting the global environment.
3. **Pull Request to Mathlib:** Once the proof is complete, I will structure the file to meet mathlib's strict formatting and documentation guidelines, with the ultimate goal of submitting a PR to integrate Simpson's Rule into the official Lean 4 mathematics library.